

Clinic Management System — Tech Lead Refactored Guide

Scope of changes: Laravel Passport (replaces Sanctum), aggressive Redis caching with tag-based invalidation, background jobs for all async/heavy work, and database query improvements (indexes, column selection, atomic locks, transactions). Only files that changed or are new are included. Everything else from the original guide stays as-is.

Code Review Summary

Area	Problem in Original	Fix Applied
Auth	Sanctum uses opaque DB tokens — no JWT, no scopes, no expiry enforcement	Replaced with Passport JWT access tokens (OAuth2 JSON role)
Booking race condition	<code>getAvailableSlots()</code> and <code>create()</code> are two separate steps — two concurrent requests can both pass the availability check and attempt to book the same slot. The DB unique constraint catches this but returns a 500, not a clean 422	<code>Cache::lock()</code> Redis around the check-then-write
No transactions	<code>PrescriptionRepository::create/update</code> writes to two tables without a transaction — a failure halfway leaves orphaned data	<code>DB::transaction()</code> multi-table writes
N+1 in relations	<code>with('patient', 'doctor', 'bookedBy')</code> loads all columns on related models	<code>with('patient:id,r', 'doctor:id,name')</code> eager loading
SELECT * on all list queries	Repositories return full model hydration even for paginated list views	Explicit <code>select([...])</code> list query
LIKE '%search%'	Leading wildcard disables the index — full table scan on patient search	MySQL FULLTEXT index <code>whereFullText()</code>
Dashboard on every request	Super admin stats execute 5 separate aggregate queries per HTTP request	Single raw aggregate query Redis cache, refreshed by background job
No schedule caching	<code>findForDayAndDoctor()</code> hits the DB on every slot calculation and every booking	Cache the full schedule for each doctor (1-hour TTL), key in memory
No cache for <code>ClinicScopeMiddleware</code>	Every API request loads <code>\$user->clinic</code> from the DB	Cache the clinic active clinic (5-min TTL), invalidated

Area	Problem in Original	Fix Applied
		clinic is toggled
No cache invalidation strategy	<code>Cache::forget()</code> calls are scattered as raw strings	Centralized <code>CacheService</code> keys and TTLs
No async side effects	Booking, prescription creation fire nothing — emails/reminders/stats are all synchronous	Background jobs for reminders, stats refresh, up
Missing DB indexes	Hot query paths have no composite indexes	New index migration columns

Step 1 — Install Laravel Passport

```
bash

# Remove Sanctum, add Passport
composer remove laravel/sanctum
composer require laravel/passport

# Publish and run Passport migrations (creates oauth_* tables)
php artisan vendor:publish --tag=passport-migrations
php artisan migrate

# Generate encryption keys (creates storage/oauth-private.key, storage/oauth-public.key)
php artisan passport:install --uuids
```

`--uuids` makes token IDs UUIDs instead of integers — better for distributed systems.

Step 2 — `config/auth.php` (updated)

```
php
```

```
'defaults' => [  
    'guard'      => 'api',  
    'passwords' => 'users',  
],  
  
'guards' => [  
    'web' => [  
        'driver'    => 'session',  
        'provider' => 'users',  
    ],  
    // ↓ Changed from 'sanctum' to 'passport'  
    'api' => [  
        'driver'    => 'passport',  
        'provider' => 'users',  
    ],  
],
```

Step 3 — `app/Providers/AppServiceProvider.php` (updated)

Configure Passport token expiry and register it:

```
php
```

```
<?php
```

```
namespace App\Providers;
```

```
use Illuminate\Support\ServiceProvider;
```

```
use Laravel\Passport\Passport;
```

```
class AppServiceProvider extends ServiceProvider
```

```
{  
    public function boot(): void  
    {  
        // Personal access tokens expire in 24 hours  
        // Refresh tokens last 30 days  
        Passport::personalAccessTokensExpireIn(now()->addHours(24));  
        Passport::refreshTokensExpireIn(now()->addDays(30));  
  
        // Restrict token scopes to roles – prevents scope escalation  
        Passport::tokensCan([  
            'super_admin' => 'Super administrator access',  
            'doctor'      => 'Doctor access',  
            'assistant'   => 'Doctor assistant access',  
        ]);  
    }  
}
```

Step 4 — `bootstrap/app.php` (updated)

```
php
```

```

->withMiddleware(function (Middleware $middleware) {
    $middleware->alias([
        'role' => \App\Http\Middleware\RoleMiddleware::class,
        'clinic.scope' => \App\Http\Middleware\ClinicScopeMiddleware::class,
    ]);
})
->withExceptions(function (Exceptions $exceptions) {
    $exceptions->render(function (\Illuminate\Validation\ValidationException $e, $request) {
        if ($request->expectsJson()) {
            return response()->json([
                'success' => false,
                'message' => 'Validation failed',
                'errors' => $e->errors(),
            ], 422);
        }
    });

    $exceptions->render(function (\Illuminate\Auth\AuthenticationException $e, $request) {
        if ($request->expectsJson()) {
            return response()->json(['success' => false, 'message' => 'Unauthenticated'], 401);
        }
    });

    // Catch DB unique constraint violations from double-booking race conditions
    // that slip past the Redis lock (e.g., lock expired under extreme load)
    $exceptions->render(function (\Illuminate\Database\QueryException $e, $request) {
        if ($request->expectsJson() && $e->errorInfo[1] === 1062) {
            return response()->json([
                'success' => false,
                'message' => 'This time slot was just taken. Please select another.'
            ], 422);
        }
    });
});
})

```

Step 5 — `app/Models/User.php` (updated)

php

```
<?php
```

```
namespace App\Models;

use App\Enums\UserRole;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Laravel\Passport\HasApiTokens; // ← Changed from Laravel\Sanctum\HasApiTokens

class User extends Authenticatable
{
    use HasApiTokens; // Now Passport's HasApiTokens – provides createToken(), token

    protected $fillable = ['clinic_id', 'name', 'email', 'password', 'phone', 'role'];

    protected $hidden = ['password', 'remember_token'];

    protected $casts = [
        'role'      => UserRole::class,
        'is_active' => 'boolean',
    ];

    public function clinic()
    {
        return $this->belongsTo(Clinic::class);
    }

    public function schedules()
    {
        return $this->hasMany(DoctorSchedule::class, 'doctor_id');
    }

    public function appointments()
    {
        return $this->hasMany(Appointment::class, 'doctor_id');
    }
}
```

Step 6 — New Index Migration

Create a dedicated migration for indexes so the original schema migrations stay clean:

```
bash
```

```
php artisan make:migration add_performance_indexes_to_all_tables
```



```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        // — appointments —————
        // Hot path: assistant loads today's clinic appointments (filtered by clinic)
        Schema::table('appointments', function (Blueprint $table) {
            $table->index(['clinic_id', 'appointment_date', 'status'], 'idx_appt_clinic_date_status');
            // Hot path: doctor loads their own appointments filtered by date
            $table->index(['doctor_id', 'appointment_date'], 'idx_appt_doctor_date');
            // Hot path: patient history lookup
            $table->index(['patient_id', 'appointment_date'], 'idx_appt_patient_date');
        });

        // — patients —————
        // Hot path: assistant searches patients in their clinic
        Schema::table('patients', function (Blueprint $table) {
            $table->index(['clinic_id', 'name'], 'idx_patient_clinic_name');
            $table->index(['clinic_id', 'phone'], 'idx_patient_clinic_phone');
            // Full-text index for MATCH...AGAINST search (replaces slow LIKE '%X%')
            $table->fullText(['name', 'phone'], 'ft_patient_name_phone');
        });

        // — doctor_schedules —————
        // Hot path: slot computation queries this on every booking and slot fetch
        Schema::table('doctor_schedules', function (Blueprint $table) {
            $table->index(['doctor_id', 'day_of_week', 'is_active'], 'idx_schedule_doctor_day_is_active');
        });

        // — prescriptions —————
        Schema::table('prescriptions', function (Blueprint $table) {
            $table->index(['doctor_id', 'clinic_id'], 'idx_prescription_doctor_clinic');
            $table->index(['patient_id'], 'idx_prescription_patient');
        });
    }

    public function down(): void
    {
        Schema::table('appointments', fn($t) => $t->dropIndex('idx_appt_clinic_date_status'));
        Schema::table('appointments', fn($t) => $t->dropIndex('idx_appt_doctor_date'));
        Schema::table('appointments', fn($t) => $t->dropIndex('idx_appt_patient_date'));
        Schema::table('patients', fn($t) => $t->dropIndex('idx_patient_clinic_name'));
        Schema::table('patients', fn($t) => $t->dropIndex('idx_patient_clinic_phone'));
        Schema::table('patients', fn($t) => $t->dropIndex('ft_patient_name_phone'));
        Schema::table('doctor_schedules', fn($t) => $t->dropIndex('idx_schedule_doctor_day_is_active'));
    }
}
```



```
Schema::table('appointments',    fn($t) => $t->dropIndex('idx_appt_patient_d
Schema::table('patients',        fn($t) => $t->dropIndex('idx_patient_clinic
Schema::table('patients',        fn($t) => $t->dropIndex('idx_patient_clinic
Schema::table('patients',        fn($t) => $t->dropFullText('ft_patient_name
Schema::table('doctor_schedules',fn($t) => $t->dropIndex('idx_schedule_docto
Schema::table('prescriptions',   fn($t) => $t->dropIndex('idx_prescription_d
Schema::table('prescriptions',   fn($t) => $t->dropIndex('idx_prescription_p
    }
};
```

Step 7 — `app/Services/CacheService.php` (NEW)

Centralizing cache keys prevents typos and makes invalidation auditable.

Never write a raw cache key string outside this class.

php

```
<?php
```

```
namespace App\Services;
```

```
use Illuminate\Support\Facades\Cache;
```

```
class CacheService
```

```
{
```

```
    // TTLs in seconds – all in one place, easy to tune
```

```
    public const TTL_SLOTS          = 300;    // 5 min – changes with each booking
```

```
    public const TTL_SCHEDULE       = 3600;   // 1 hour – doctors rarely update sched
```

```
    public const TTL_STATS          = 900;    // 15 min – dashboard aggregate stats
```

```
    public const TTL_CLINIC_ACTIVE  = 300;    // 5 min – per-user clinic status chec
```

```
    public const TTL_PATIENT_LIST   = 600;    // 10 min – paginated patient list
```

```
    // — Key builders —————
```

```
    public static function slotsKey(int $doctorId, string $date): string
```

```
    {
```

```
        return "slots:doctor:{$doctorId}:date:{$date}";
```

```
    }
```

```
    public static function bookingLockKey(int $doctorId, string $date, string $start
```

```
    {
```

```
        return "lock:booking:{$doctorId}:{$date}:{$startTime}";
```

```
    }
```

```
    public static function scheduleKey(int $doctorId): string
```

```
    {
```

```
        return "schedule:doctor:{$doctorId}:all";
```

```
    }
```

```
    public static function userClinicActiveKey(int $userId): string
```

```
    {
```

```
        return "user:{$userId}:clinic_active";
```

```
    }
```

```
    public static function dashboardStatsKey(): string
```

```
    {
```

```
        return 'dashboard:super_admin:stats';
```

```
    }
```

```
    public static function patientListKey(int $clinicId, string $search, int $page):
```

```
    {
```

```
        $searchHash = md5($search); // avoid special chars in key
```

```
        return "patients:clinic:{$clinicId}:q:{$searchHash}:page:{$page}";
```

```
}

// — Invalidation helpers —————

/**
 * Flush everything scoped to a clinic (tagged cache).
 * Call this when a clinic is toggled active/inactive or updated.
 */
public static function flushClinic(int $clinicId): void
{
    Cache::tags(["clinic:{$clinicId}"])->flush();
}

/**
 * Flush all doctor schedules from cache and the derived slot cache.
 * Call this when a DoctorSchedule is created, updated, or deleted.
 */
public static function flushDoctorSchedule(int $doctorId): void
{
    Cache::tags(["doctor:{$doctorId}:schedule"])->flush();
    Cache::tags(["doctor:{$doctorId}:slots"])->flush();
}

/**
 * Flush a specific slot date – called after a booking, reschedule, or cancel.
 */
public static function flushSlots(int $doctorId, string $date): void
{
    Cache::forget(self::slotsKey($doctorId, $date));
}

/**
 * Flush a specific user's clinic-active status cache.
 * Call this when a user is toggled or their clinic is toggled.
 */
public static function flushUserClinicActive(int $userId): void
{
    Cache::forget(self::userClinicActiveKey($userId));
}

/**
 * Flush dashboard stats. Called after any write that changes system counts.
 */
public static function flushDashboardStats(): void
{
    Cache::forget(self::dashboardStatsKey());
}
```

```
}  
}
```

Step 8 — `app/Http/Middleware/ClinicScopeMiddleware.php` (updated)

Original hit the DB on every single authenticated request to check clinic status.

Now: cached per user with a 5-minute TTL and invalidated on toggle.

```
php
```

```
<?php
```

```
namespace App\Http\Middleware;
```

```
use App\Services\CacheService;
```

```
use Closure;
```

```
use Illuminate\Http\Request;
```

```
use Illuminate\Support\Facades\Cache;
```

```
class ClinicScopeMiddleware
```

```
{
```

```
    public function handle(Request $request, Closure $next)
```

```
    {
```

```
        $user = $request->user();
```

```
        if ($user->role->value === 'super_admin') {
```

```
            return $next($request);
```

```
        }
```

```
        // Cache the active check – avoids a DB join on every API call.
```

```
        // Invalidated by InvalidateClinicCacheJob when the clinic is toggled.
```

```
        $cacheKey = CacheService::userClinicActiveKey($user->id);
```

```
        $clinicId = Cache::remember($cacheKey, CacheService::TTL_CLINIC_ACTIVE, func
```

```
            // Load clinic only if not already in memory
```

```
            $user->loadMissing('clinic');
```

```
            if (! $user->is_active || ! $user->clinic?->is_active) {
```

```
                return null; // Store null to represent "inactive"
```

```
            }
```

```
            return $user->clinic_id;
```

```
        });
```

```
        if ($clinicId === null) {
```

```
            return response()->json([
```

```
                'success' => false,
```

```
                'message' => 'Your account or clinic is inactive.',
```

```
            ], 403);
```

```
        }
```

```
        $request->merge(['clinic_id' => $clinicId]);
```

```
        return $next($request);
```

```
}  
}
```

Step 9 — `app/Services/AuthService.php` (updated)

php

```
<?php
```

```
namespace App\Services;
```

```
use App\Models\User;
```

```
use Illuminate\Support\Facades\Hash;
```

```
use Illuminate\Validation\ValidationException;
```

```
class AuthService
```

```
{
```

```
    public function login(string $email, string $password): array
```

```
    {
```

```
        $user = User::where('email', $email)
```

```
            ->select(['id', 'clinic_id', 'name', 'email', 'password', 'role', 'is_active'])
```

```
            ->first();
```

```
        if (! $user || ! Hash::check($password, $user->password)) {
```

```
            throw ValidationException::withMessages([
```

```
                'email' => ['The provided credentials are incorrect.'],
```

```
            ]);
```

```
        }
```

```
        if (! $user->is_active) {
```

```
            throw ValidationException::withMessages([
```

```
                'email' => ['Your account has been deactivated.'],
```

```
            ]);
```

```
        }
```

```
        // Passport personal access token scoped to the user's role.
```

```
        // The scope is checked downstream by RoleMiddleware.
```

```
        // $tokenResult->accessToken is a JWT string – stateless, cryptographically
```

```
$tokenResult = $user->createToken('clinic-api', [$user->role->value]);
```

```
        return [
```

```
            'user' => $user,
```

```
            'token' => $tokenResult->accessToken, // JWT string
```

```
            'expires_at' => $tokenResult->token->expires_at->toISOString(),
```

```
        ];
```

```
    }
```

```
    public function logout(User $user): void
```

```
    {
```

```
        // Revoke the current OAuth token – it's immediately invalid.
```

```
        // With Sanctum, you deleted a DB row. With Passport, the JWT is
```

```
        // also revoked in the oauth_access_tokens table.
```

```
$user->token()->revoke();
```

```
}

public function logoutAll(User $user): void
{
    // Revoke ALL tokens (e.g., "sign out from all devices")
    $user->tokens()->update(['revoked' => true]);
}
}
```

Step 10 — `app/Repositories/ScheduleRepository.php` (updated)

The original called a DB query every time a slot was calculated or a booking was made. Now the entire schedule collection is cached per doctor and filtered in memory.

php


```
<?php
```

```
namespace App\Repositories;
```

```
use App\Models\DoctorSchedule;
```

```
use App\Services\CacheService;
```

```
use Illuminate\Support\Collection;
```

```
use Illuminate\Support\Facades\Cache;
```

```
class ScheduleRepository
```

```
{  
    public function allForDoctor(int $doctorId)  
    {  
        return DoctorSchedule::where('doctor_id', $doctorId)->get();  
    }  
  
    public function find(int $id): DoctorSchedule  
    {  
        return DoctorSchedule::findOrFail($id);  
    }  
  
    public function create(array $data): DoctorSchedule  
    {  
        $schedule = DoctorSchedule::create($data);  
        // Schedule changed – flush the cached collection for this doctor  
        CacheService::flushDoctorSchedule($data['doctor_id']);  
        return $schedule;  
    }  
  
    public function update(DoctorSchedule $schedule, array $data): DoctorSchedule  
    {  
        $schedule->update($data);  
        CacheService::flushDoctorSchedule($schedule->doctor_id);  
        return $schedule;  
    }  
  
    public function delete(DoctorSchedule $schedule): void  
    {  
        $doctorId = $schedule->doctor_id;  
        $schedule->delete();  
        CacheService::flushDoctorSchedule($doctorId);  
    }  
  
    /**  
     * Returns the active schedule for a given doctor and day.  
     *  
     */  
}
```

```

* IMPROVEMENT: instead of one DB query per day (as in the original),
* we cache ALL schedules for this doctor as a keyed collection,
* then filter by day_of_week in memory. One DB hit, then pure cache.
*/
public function findForDayAndDoctor(int $doctorId, int $dayOfWeek): ?DoctorSchedule
{
    $schedules = $this->getAllActiveSchedulesForDoctor($doctorId);
    return $schedules->get($dayOfWeek);
}

/**
 * Returns a Collection keyed by day_of_week for O(1) lookup.
 * Cached under a tagged key so it can be flushed when the schedule changes.
 */
public function getAllActiveSchedulesForDoctor(int $doctorId): Collection
{
    return Cache::tags(["doctor:{$doctorId}:schedule"])
        ->remember(
            CacheService::scheduleKey($doctorId),
            CacheService::TTL_SCHEDULE,
            fn() => DoctorSchedule::select(['id', 'doctor_id', 'day_of_week', 'is_active']
                ->where('doctor_id', $doctorId)
                ->where('is_active', true)
                ->get()
                ->keyBy('day_of_week') // Collection keyed by day_of_week → O(1)
            );
}
}

```

Step 11 — `app/Repositories/AppointmentRepository.php` (updated)

Constrained eager loading (`with('relation:col1,col2')`) and explicit column selection eliminates unnecessary data transfer. Note: when constraining relation columns, always include the FK column or the relation won't bind.

php

```
<?php
```

```
namespace App\Repositories;
```

```
use App\Models\Appointment;
```

```
use Illuminate\Support\Facades\DB;
```

```
class AppointmentRepository
```

```
{  
    public function allForDoctor(int $doctorId, array $filters = [], int $perPage =  
    {  
        return Appointment::select([  
            'id', 'doctor_id', 'patient_id', 'booked_by',  
            'appointment_date', 'start_time', 'end_time', 'status', 'notes',  
        ])  
        ->with([  
            'patient:id,name,phone,gender',    // only what the list view needs  
            'bookedBy:id,name',  
        ])  
        ->where('doctor_id', $doctorId)  
        ->when(isset($filters['date']), fn($q) => $q->whereDate('appointment_d  
        ->when(isset($filters['status']), fn($q) => $q->where('status', $filters  
        ->orderByDesc('appointment_date')  
        ->paginate($perPage);  
    }  
  
    public function allForClinic(int $clinicId, array $filters = [], int $perPage =  
    {  
        return Appointment::select([  
            'id', 'clinic_id', 'doctor_id', 'patient_id', 'booked_by',  
            'appointment_date', 'start_time', 'end_time', 'status',  
        ])  
        ->with([  
            'patient:id,name,phone',  
            'doctor:id,name',  
            'bookedBy:id,name',  
        ])  
        ->where('clinic_id', $clinicId)  
        ->when(isset($filters['date']), fn($q) => $q->whereDate('appointment_d  
        ->when(isset($filters['status']), fn($q) => $q->where('status', $filters  
        ->orderBy('appointment_date')  
        ->orderBy('start_time')  
        ->paginate($perPage);  
    }  
  
    public function allForAdmin(array $filters = [], int $perPage = 15)
```

```

{
    return Appointment::select([
        'id', 'clinic_id', 'doctor_id', 'patient_id',
        'appointment_date', 'start_time', 'end_time', 'status',
    ])
    ->with([
        'patient:id,name,phone',
        'doctor:id,name',
        'clinic:id,name',
    ])
    ->when(isset($filters['clinic_id']), fn($q) => $q->where('clinic_id', $f
    ->when(isset($filters['doctor_id']), fn($q) => $q->where('doctor_id', $f
    ->when(isset($filters['status']), fn($q) => $q->where('status', $filt
    ->when(isset($filters['date']), fn($q) => $q->whereDate('appointment
    ->orderByDesc('appointment_date')
    ->paginate($perPage);
}

public function find(int $id): Appointment
{
    // Full detail view – load everything including prescription
    return Appointment::with([
        'patient',
        'doctor:id,name,email,phone',
        'bookedBy:id,name',
        'prescription.items',
    ])->findOrFail($id);
}

public function create(array $data): Appointment
{
    return Appointment::create($data);
}

public function update(Appointment $appointment, array $data): Appointment
{
    $appointment->update($data);
    return $appointment;
}
}

```

Step 12 — `app/Repositories/PatientRepository.php` (updated)

Replaces `LIKE '%search%'` (full-table scan) with MySQL FULLTEXT search. Patient list results are cached per clinic + search term + page.


```
<?php
```

```
namespace App\Repositories;
```

```
use App\Models\Patient;
```

```
use App\Services\CacheService;
```

```
use Illuminate\Support\Facades\Cache;
```

```
class PatientRepository
```

```
{
```

```
    public function allForClinic(int $clinicId, ?string $search = null, int $perPage  
    {
```

```
        $page      = request()->get('page', 1);
```

```
        $cacheKey = CacheService::patientListKey($clinicId, $search ?? '', $page);
```

```
        return Cache::tags(["clinic:{$clinicId}", 'patients'])
```

```
            ->remember($cacheKey, CacheService::TTL_PATIENT_LIST, function () use ($
```

```
                return Patient::select(['id', 'clinic_id', 'name', 'phone', 'email',
```

```
                    ->where('clinic_id', $clinicId)
```

```
                    ->when($search, fn($q) => $q->whereFullText(['name', 'phone'], $
```

```
                    ->orderBy('name')
```

```
                    ->paginate($perPage);
```

```
            });
```

```
    }
```

```
    public function find(int $id): Patient
```

```
    {
```

```
        return Patient::with([
```

```
            'appointments' => fn($q) => $q->select(['id', 'patient_id', 'doctor_id',
```

```
                ->with('prescription:id,appointment_id,diagnosis')
```

```
                ->orderByDesc('appointment_date')
```

```
                ->limit(20), // don't load unlimited history
```

```
        ])->findOrFail($id);
```

```
    }
```

```
    public function create(array $data): Patient
```

```
    {
```

```
        $patient = Patient::create($data);
```

```
        // New patient – flush the clinic's patient list cache
```

```
        Cache::tags(["clinic:{$data['clinic_id']}", 'patients'])->flush();
```

```
        return $patient;
```

```
    }
```

```
    public function update(Patient $patient, array $data): Patient
```

```
    {
```

```
        $patient->update($data);
```

```
        Cache::tags(["clinic:{$patient->clinic_id}", 'patients'])->flush();
        return $patient;
    }
}
```

Step 13 — `app/Repositories/PrescriptionRepository.php` (updated)

Both `create` and `update` now run inside `DB::transaction()`. Without this, a failure between writing the prescription header and inserting items leaves a prescription with no items — silent corrupt data.

php

```
<?php
```

```
namespace App\Repositories;
```

```
use App\Models\Prescription;
```

```
use Illuminate\Support\Facades\DB;
```

```
class PrescriptionRepository
```

```
{
```

```
    public function allForDoctor(int $doctorId, int $perPage = 15)
```

```
    {
```

```
        return Prescription::select(['id', 'doctor_id', 'patient_id', 'clinic_id', '
            ->with(['patient:id,name,phone', 'items:id,prescription_id,medicine_name
            ->where('doctor_id', $doctorId)
            ->orderByDesc('created_at')
            ->paginate($perPage);
```

```
    }
```

```
    public function allForAdmin(int $perPage = 15)
```

```
    {
```

```
        return Prescription::select(['id', 'doctor_id', 'patient_id', 'clinic_id', '
            ->with([
                'patient:id,name,phone',
                'doctor:id,name',
                'clinic:id,name',
                'items:id,prescription_id,medicine_name,dosage',
            ])
            ->orderByDesc('created_at')
            ->paginate($perPage);
```

```
    }
```

```
    public function find(int $id): Prescription
```

```
    {
```

```
        return Prescription::with(['patient', 'items', 'appointment:id,appointment_d
            ->findOrFail($id);
```

```
    }
```

```
/**
```

```
 * Wrapped in a transaction. If `createMany` fails, the prescription header
 * is also rolled back – no orphaned rows.
```

```
*/
```

```
    public function create(array $data, array $items): Prescription
```

```
    {
```

```
        return DB::transaction(function () use ($data, $items) {
            $prescription = Prescription::create($data);
            $prescription->items()->createMany($items);
```



```

        return $prescription->load('items');
    });
}

/**
 * Replaces items atomically. Without a transaction, if the app crashes
 * between delete() and createMany(), the prescription has zero items.
 */
public function update(Prescription $prescription, array $data, array $items): Prescription
{
    return DB::transaction(function () use ($prescription, $data, $items) {
        $prescription->update($data);
        $prescription->items()->delete();
        $prescription->items()->createMany($items);
        return $prescription->load('items');
    });
}
}

```

Step 14 — `app/Repositories/DashboardRepository.php` (NEW)

Original had multiple round-trips to the DB for the super admin dashboard.

This runs a single raw aggregate query and caches the result.

php

```
<?php
```

```
namespace App\Repositories;
```

```
use App\Services\CacheService;
```

```
use Illuminate\Support\Facades\Cache;
```

```
use Illuminate\Support\Facades\DB;
```

```
class DashboardRepository
```

```
{
```

```
    /**
```

```
     * Returns all system-wide counts in ONE database round-trip.
```

```
     * Cached for TTL_STATS seconds. Refreshed by GenerateDashboardStatsJob.
```

```
     *
```

```
     * Original: 5 separate Eloquent calls = 5 DB queries per page load.
```

```
     * Refactored: 1 raw query + Redis cache.
```

```
    */
```

```
public function superAdminStats(): array
```

```
{
```

```
    return Cache::remember(
```

```
        CacheService::dashboardStatsKey(),
```

```
        CacheService::TTL_STATS,
```

```
        fn() => $this->computeSuperAdminStats()
```

```
    );
```

```
}
```

```
public function computeSuperAdminStats(): array
```

```
{
```

```
    // All counts in one query – subqueries are optimized by the MySQL planner
```

```
    $row = DB::selectOne("
```

```
        SELECT
```

```
            (SELECT COUNT(*) FROM clinics WHERE deleted_at IS NULL)
```

```
            (SELECT COUNT(*) FROM users WHERE role = 'doctor')
```

```
            (SELECT COUNT(*) FROM users WHERE role = 'assistant')
```

```
            (SELECT COUNT(*) FROM patients)
```

```
            (SELECT COUNT(*) FROM appointments)
```

```
            (SELECT COUNT(*) FROM appointments WHERE status = 'pending')
```

```
            (SELECT COUNT(*) FROM appointments WHERE status = 'completed')
```

```
            (SELECT COUNT(*) FROM appointments WHERE appointment_date = CURDATE()
```

```
        ");
```

```
    return (array) $row;
```

```
}
```

```
    /**
```

```
     * Returns per-clinic summary for a doctor's dashboard.
```

```
* These are simpler and can use Eloquent.  
*/  
public function doctorStats(int $doctorId): array  
{  
    return DB::selectOne("  
        SELECT  
            COUNT(CASE WHEN appointment_date = CURDATE() THEN 1 END)  
            COUNT(CASE WHEN status = 'pending' AND appointment_date >= CURDATE()  
            COUNT(CASE WHEN status = 'completed' THEN 1 END)  
        FROM appointments  
        WHERE doctor_id = ?  
        ", [$doctorId]);  
}  
}
```

Step 15 — `app/Services/AppointmentService.php` (updated)

Three critical improvements:

1. **Redis atomic lock** — prevents race conditions between concurrent booking requests
2. `DB::transaction()` — the write is atomic
3. **Job dispatch** — side effects are async, not blocking the HTTP response

php

```
<?php
```

```
namespace App\Services;
```

```
use App\Enums\AppointmentStatus;
use App\Jobs\SendAppointmentConfirmationJob;
use App\Jobs\WarmSlotCacheJob;
use App\Models\Appointment;
use App\Repositories\AppointmentRepository;
use App\Repositories\ScheduleRepository;
use App\Services\CacheService;
use Carbon\Carbon;
use Illuminate\Support\Facades\Cache;
use Illuminate\Support\Facades\DB;
use Illuminate\Validation\ValidationException;
```

```
class AppointmentService
```

```
{
    public function __construct(
        private AppointmentRepository $appointmentRepo,
        private ScheduleRepository $scheduleRepo,
    ) {}

    /**
     * Returns available HH:MM slots for a doctor on a date.
     * Cached under a tagged key so booking/cancel can flush exactly this date.
     */
    public function getAvailableSlots(int $doctorId, string $date): array
    {
        $cacheKey = CacheService::slotsKey($doctorId, $date);

        return Cache::tags(["doctor:{$doctorId}:slots"])
            ->remember($cacheKey, CacheService::TTL_SLOTS, function () use ($doctorId, $date) {
                $dayOfWeek = Carbon::parse($date)->dayOfWeek;
                $schedule = $this->scheduleRepo->findForDayAndDoctor($doctorId, $date);

                if (! $schedule) {
                    return [];
                }

                // Pull only start_times – no need to hydrate full Appointment model
                $bookedTimes = Appointment::where('doctor_id', $doctorId)
                    ->whereDate('appointment_date', $date)
                    ->whereNotIn('status', ['cancelled'])
                    ->pluck('start_time')
                    ->map(fn($t) => substr($t, 0, 5)) // normalize "09:00:00" → "09:"
            });
    }
}
```

```

        ->toArray());

        $slots    = [];
        $current  = Carbon::parse("{\$date} {\$schedule->start_time}");
        $end      = Carbon::parse("{\$date} {\$schedule->end_time}");
        $duration = \$schedule->slot_duration;

        while ($current->copy()->addMinutes($duration)->lte($end)) {
            $slotTime = $current->format('H:i');
            if (! in_array($slotTime, $bookedTimes)) {
                $slots[] = $slotTime;
            }
            $current->addMinutes($duration);
        }

        return $slots;
    });
}

/**
 * Books an appointment atomically.
 *
 * RACE CONDITION FIX: Without the Redis lock, two concurrent requests can
 * both call getAvailableSlots(), both find the slot free, and both try to
 * insert – one succeeds, the other crashes with a QueryException on the
 * unique constraint. The lock serializes concurrent bookings for the same
 * slot so only one proceeds; the other gets a clean 422.
 */
public function book(array $data): Appointment
{
    $doctorId = $data['doctor_id'];
    $date     = $data['appointment_date'];
    $start    = $data['start_time'];

    // 10-second lock scoped to the exact slot being booked
    $lock = Cache::lock(CacheService::bookingLockKey($doctorId, $date, $start),

    if (! $lock->get()) {
        // Another request is actively booking this slot right now
        throw ValidationException::withMessages([
            'start_time' => ['This slot is being processed. Please try again.'],
        ]);
    }

    try {
        $availableSlots = $this->getAvailableSlots($doctorId, $date);
    }

```

```

        if (! in_array($start, $availableSlots)) {
            throw ValidationException::withMessages([
                'start_time' => ['This time slot is not available.'],
            ]);
        }

        $dayOfWeek = Carbon::parse($date)->dayOfWeek;
        $schedule = $this->scheduleRepo->findForDayAndDoctor($doctorId, $dayOfWeek);
        $data['end_time'] = Carbon::parse($start)->addMinutes($schedule->slot_duration);

        // DB::transaction ensures the insert is atomic
        $appointment = DB::transaction(fn() => $this->appointmentRepo->create($data));

        // Flush the slot cache for this date immediately after booking
        CacheService::flushSlots($doctorId, $date);

        // Send confirmation asynchronously – does NOT block the HTTP response
        SendAppointmentConfirmationJob::dispatch($appointment)->onQueue('notifications');

        return $appointment;
    } finally {
        // Always release the lock even if an exception was thrown
        $lock->release();
    }
}

public function reschedule(Appointment $appointment, array $data): Appointment
{
    if ($appointment->status === AppointmentStatus::Completed) {
        throw ValidationException::withMessages([
            'status' => ['A completed appointment cannot be rescheduled.'],
        ]);
    }

    $doctorId = $appointment->doctor_id;
    $newDate = $data['appointment_date'];
    $newStart = $data['start_time'];
    $oldDate = $appointment->appointment_date->format('Y-m-d');

    $lock = Cache::lock(CacheService::bookingLockKey($doctorId, $newDate, $newStart));

    if (!$lock->get()) {
        throw ValidationException::withMessages([
            'start_time' => ['This slot is being processed. Please try again.'],
        ]);
    }
}

```

```

try {
    $availableSlots = $this->getAvailableSlots($doctorId, $newDate);

    if (! in_array($newStart, $availableSlots)) {
        throw ValidationException::withMessages([
            'start_time' => ['This time slot is not available.'],
        ]);
    }

    $dayOfWeek = Carbon::parse($newDate)->dayOfWeek;
    $schedule = $this->scheduleRepo->findForDayAndDoctor($doctorId, $dayOfWeek);
    $data['end_time'] = Carbon::parse($newStart)->addMinutes($schedule->slotDuration);

    $updated = DB::transaction(fn() => $this->appointmentRepo->update($appointment, $data));

    // Flush both the old date (slot freed) and the new date (slot taken)
    CacheService::flushSlots($doctorId, $oldDate);
    CacheService::flushSlots($doctorId, $newDate);

    return $updated;
} finally {
    $lock->release();
}
}

public function cancel(Appointment $appointment): Appointment
{
    $updated = $this->appointmentRepo->update($appointment, [
        'status' => AppointmentStatus::Cancelled->value,
    ]);

    // Freed slot – flush so it reappears in available slots immediately
    CacheService::flushSlots($appointment->doctor_id, $appointment->appointment_start);

    return $updated;
}

public function updateStatus(Appointment $appointment, string $status): Appointment
{
    return $this->appointmentRepo->update($appointment, ['status' => $status]);
}
}

```

Step 16 — `app/Services/ClinicService.php` (updated)

When a clinic is toggled, we need to flush all per-user clinic_active caches for its members, plus all clinic-tagged data. We do this via a background job.

```
php

<?php

namespace App\Services;

use App\Jobs\InvalidateClinicCacheJob;
use App\Models\Clinic;
use App\Repositories\ClinicRepository;

class ClinicService
{
    public function __construct(private ClinicRepository $repo) {}

    public function toggle(Clinic $clinic): Clinic
    {
        $updated = $this->repo->toggle($clinic);

        // Async cache invalidation - flush clinic cache + per-user clinic_active
        InvalidateClinicCacheJob::dispatch($clinic->id)->onQueue('cache');

        return $updated;
    }
}
```

Step 17 — Background Jobs

`app/Jobs/SendAppointmentConfirmationJob.php`

```
php
```



```
<?php
```

```
namespace App\Jobs;
```

```
use App\Models\Appointment;
```

```
use Illuminate\Bus\Queueable;
```

```
use Illuminate\Contracts\Queue\ShouldQueue;
```

```
use Illuminate\Foundation\Bus\Dispatchable;
```

```
use Illuminate\Queue\InteractsWithQueue;
```

```
use Illuminate\Queue\SerializesModels;
```

```
use Illuminate\Support\Facades\Log;
```

```
class SendAppointmentConfirmationJob implements ShouldQueue
```

```
{  
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;
```

```
    // Retry up to 3 times with exponential backoff
```

```
    public int $tries = 3;
```

```
    public int $backoff = 10; // seconds between retries
```

```
    public function __construct(private Appointment $appointment) {}
```

```
    public function handle(): void  
{
```

```
        $appointment = $this->appointment->load(['patient', 'doctor:id,name']);
```

```
        // Swap this Log::info() for Mail::to(...)->send(new AppointmentConfirmed($a  
        // when mail is configured. The job contract stays the same.
```

```
        Log::channel('appointments')->info('Appointment confirmation', [  
            'appointment_id' => $appointment->id,  
            'patient'        => $appointment->patient->name,  
            'doctor'         => $appointment->doctor->name,  
            'date'           => $appointment->appointment_date->format('Y-m-d'),  
            'time'           => $appointment->start_time,  
        ]);  
    }
```

```
    /**
```

```
     * If the job exhausts retries, log to the failed_jobs table.
```

```
     * Never let a failed notification crash a booking.
```

```
    */
```

```
    public function failed(\Throwable $exception): void  
{
```

```
        Log::error('AppointmentConfirmation job failed', [  
            'appointment_id' => $this->appointment->id,  
            'error'          => $exception->getMessage(),  
        ];
```

```
        ]);  
    }  
}
```

app/Jobs/WarmSlotCacheJob.php

Dispatched whenever a doctor's schedule is created, updated, or deleted.
Pre-computes available slots for the next N days so the first request after
a schedule change hits the cache, not the DB.

php

```
<?php
```

```
namespace App\Jobs;

use App\Services\AppointmentService;
use Carbon\Carbon;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

class WarmSlotCacheJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public int $tries = 2;

    public function __construct(
        private int $doctorId,
        private int $daysAhead = 14
    ) {}

    /**
     * Pre-populates the slot cache for this doctor for the next $daysAhead days.
     * Each call to getAvailableSlots() will either read from cache or compute + cac
     */
    public function handle(AppointmentService $service): void
    {
        $today = Carbon::today();

        for ($i = 0; $i < $this->daysAhead; $i++) {
            $date = $today->copy()->addDays($i)->format('Y-m-d');
            $service->getAvailableSlots($this->doctorId, $date);
        }
    }
}
```

Dispatch it from **ScheduleService** after any create/update/delete:

```
php
```

```
// In app/Services/ScheduleService.php – add after each mutation:
WarmSlotCacheJob::dispatch($schedule->doctor_id)->onQueue('cache');
```

app/Jobs/GenerateDashboardStatsJob.php

Refreshes the super admin stats cache on a schedule.
The scheduler fires this every 15 minutes — see Step 18.

php

<?php

```
namespace App\Jobs;

use App\Repositories\DashboardRepository;
use App\Services\CacheService;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Cache;

class GenerateDashboardStatsJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public int $tries = 2;

    public function handle(DashboardRepository $repo): void
    {
        // Compute fresh stats and overwrite the cache
        $stats = $repo->computeSuperAdminStats();

        Cache::put(
            CacheService::dashboardStatsKey(),
            $stats,
            CacheService::TTL_STATS
        );
    }
}
```

app/Jobs/SendAppointmentReminderJob.php

Sent 24 hours before each appointment. The scheduler (Step 18) dispatches one of these per upcoming appointment daily.

php

```
<?php
```

```
namespace App\Jobs;

use App\Models\Appointment;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;
use Illuminate\Support\Facades\Log;

class SendAppointmentReminderJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public int $tries = 3;
    public int $backoff = 30;

    public function __construct(private Appointment $appointment) {}

    public function handle(): void
    {
        $appointment = $this->appointment->load(['patient', 'doctor:id,name']);

        // Replace with Mail::to($appointment->patient->email)->send(new Appointment
        Log::channel('appointments')->info('Appointment reminder sent', [
            'appointment_id' => $appointment->id,
            'patient'        => $appointment->patient->name,
            'date'           => $appointment->appointment_date->format('Y-m-d'),
        ]));
    }
}
```

app/Jobs/InvalidateClinicCacheJob.php

Flushes all cached data for a clinic and every user in it.
Triggered when a clinic is toggled active/inactive.

```
php
```

```
<?php
```

```
namespace App\Jobs;

use App\Models\User;
use App\Services\CacheService;
use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Bus\Dispatchable;
use Illuminate\Queue\InteractsWithQueue;
use Illuminate\Queue\SerializesModels;

class InvalidateClinicCacheJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function __construct(private int $clinicId) {}

    public function handle(): void
    {
        // 1. Flush all clinic-tagged data (patient lists, appointments, etc.)
        CacheService::flushClinic($this->clinicId);

        // 2. Flush the per-user clinic_active check for every member
        //     Use select('id') + each() to avoid loading full User models in memory
        User::where('clinic_id', $this->clinicId)
            ->select('id')
            ->each(fn($u) => CacheService::flushUserClinicActive($u->id));
    }
}
```

Step 18 — Scheduler (`routes/console.php`)

```
php
```

```
<?php
```

```
use App\Jobs\GenerateDashboardStatsJob;
use App\Jobs\SendAppointmentReminderJob;
use App\Models\Appointment;
use Illuminate\Support\Facades\Schedule;

// Refresh super admin dashboard stats every 15 minutes
Schedule::job(new GenerateDashboardStatsJob, 'cache')
    ->everyFifteenMinutes()
    ->withoutOverlapping(); // skip if a previous run is still processing

// Every morning at 08:00, send reminders for tomorrow's appointments.
// Chunked so we don't load thousands of rows into memory at once.
Schedule::call(function () {
    $tomorrow = now()->addDay()->format('Y-m-d');

    Appointment::whereDate('appointment_date', $tomorrow)
        ->whereIn('status', ['pending', 'confirmed'])
        ->select(['id', 'doctor_id', 'patient_id', 'appointment_date', 'start_time'])
        ->chunk(100, function ($appointments) {
            foreach ($appointments as $appointment) {
                SendAppointmentReminderJob::dispatch($appointment)
                    ->onQueue('notifications');
            }
        });
    });
})->dailyAt('08:00')->name('dispatch-appointment-reminders')->withoutOverlapping();
```

Step 19 — `routes/api.php` (updated)

One change only: `auth:sanctum` → `auth:api` everywhere.

```
php
```

<?php

```
use App\Http\Controllers\Auth\AuthController;
// ... all other imports unchanged ...
use Illuminate\Support\Facades\Route;

// — Auth —————
Route::prefix('auth')->group(function () {
    Route::post('login', [AuthController::class, 'login'])
        ->middleware('throttle:5,1');

    Route::middleware('auth:api')->group(function () { // ← was auth:sanctum
        Route::post('logout', [AuthController::class, 'logout']);
        Route::get('me', [AuthController::class, 'me']);
    });
});

// — Super Admin —————
Route::middleware(['auth:api', 'role:super_admin']) // ← was auth:sanctum
    ->prefix('super-admin')
    ->group(function () {
        Route::apiResource('clinics', ClinicController::class)->except(['destroy']);
        Route::patch('clinics/{clinic}/toggle', [ClinicController::class, 'toggle']);
        Route::apiResource('users', UserController::class)->except(['destroy']);
        Route::patch('users/{user}/toggle', [UserController::class, 'toggle']);
        Route::get('dashboard', [DashboardController::class, 'index']);
        Route::get('appointments', [DashboardController::class, 'appointments']);
        Route::get('prescriptions', [DashboardController::class, 'prescriptions']);
    });

// — Doctor —————
Route::middleware(['auth:api', 'role:doctor', 'clinic.scope']) // ← was auth:sanctum
    ->prefix('doctor')
    ->group(function () {
        Route::apiResource('schedules', ScheduleController::class);
        Route::get('appointments', [DoctorAppointmentController::class, 'index']);
        Route::get('appointments/{appointment}', [DoctorAppointmentController::class, 'show']);
        Route::patch('appointments/{appointment}/status', [DoctorAppointmentController::class, 'toggleStatus']);
        Route::apiResource('prescriptions', PrescriptionController::class)->except(['destroy']);
        Route::get('patients', [DoctorPatientController::class, 'index']);
        Route::get('patients/{patient}', [DoctorPatientController::class, 'show']);
    });

// — Assistant —————
Route::middleware(['auth:api', 'role:assistant', 'clinic.scope']) // ← was auth:sanctum
    ->prefix('assistant')
```



```
->group(function () {
    Route::apiResource('patients', AssistantPatientController::class)->except(['
    Route::get('appointments', [AssistantAppointmentController::cl
    Route::post('appointments', [AssistantAppointmentController::cl
    Route::get('appointments/{appointment}', [AssistantAppointmentController::cl
    Route::put('appointments/{appointment}', [AssistantAppointmentController::cl
    Route::delete('appointments/{appointment}', [AssistantAppointmentController:
    Route::get('available-slots', [AssistantAppointmentController::cl
});
```

Step 20 — Updated `.env`

```
env

APP_NAME=ClinicAPI
APP_ENV=local
APP_KEY=          # php artisan key:generate
APP_DEBUG=true
APP_URL=http://localhost


DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=clinic_api
DB_USERNAME=root
DB_PASSWORD=


CACHE_DRIVER=redis
QUEUE_CONNECTION=redis
SESSION_DRIVER=redis


REDIS_HOST=127.0.0.1
REDIS_PASSWORD=null
REDIS_PORT=6379


# Passport stores keys here – ensure these paths are writable and NOT in .gitignore
PASSPORT_PRIVATE_KEY=storage/oauth-private.key
PASSPORT_PUBLIC_KEY=storage/oauth-public.key


# Separate queues for different workload priorities
# Run these as separate worker processes with different concurrency
QUEUE_NOTIFICATIONS=notifications # appointment confirmations, reminders
QUEUE_CACHE=cache                 # slot warming, stats refresh, cache invalidatio
```

Step 21 — Queue Worker Setup

Run three separate worker processes — each handles a different priority workload:

```
bash

# Default queue — handles general jobs
php artisan queue:work redis --queue=default --tries=3 --sleep=3

# Notifications queue — patient-facing messages
# Lower concurrency but high reliability (--tries=3, --backoff=10)
php artisan queue:work redis --queue=notifications --tries=3 --sleep=1

# Cache queue — fast, idempotent jobs (slot warming, stats refresh)
# Higher concurrency is fine — these jobs are safe to run in parallel
php artisan queue:work redis --queue=cache --tries=2 --sleep=1

# Run the scheduler (once — it dispatches jobs every minute internally)
php artisan schedule:work
```

In production, use Supervisor to keep these processes alive:

```
ini
```

```
; /etc/supervisor/conf.d/clinic-workers.conf
```

```
[program:clinic-queue-default]
```

```
command=php /var/www/clinic-api/artisan queue:work redis --queue=default --tries=3
```

```
autostart=true
```

```
autorestart=true
```

```
numprocs=2
```

```
[program:clinic-queue-notifications]
```

```
command=php /var/www/clinic-api/artisan queue:work redis --queue=notifications --tries=3
```

```
autostart=true
```

```
autorestart=true
```

```
numprocs=2
```

```
[program:clinic-queue-cache]
```

```
command=php /var/www/clinic-api/artisan queue:work redis --queue=cache --tries=2
```

```
autostart=true
```

```
autorestart=true
```

```
numprocs=3
```

```
[program:clinic-scheduler]
```

```
command=php /var/www/clinic-api/artisan schedule:work
```

```
autostart=true
```

```
autorestart=true
```

```
numprocs=1
```

Step 22 — Updated Tests

Passport tokens work differently in tests. Replace `actingAs($user)` Sanctum pattern with a Passport helper, or use the `actingAs` override with the `api` guard.

tests/Feature/AuthTest.php (updated)

```
php
```

```
<?php
```

```
namespace Tests\Feature;
```

```
use App\Models\User;
```

```
use Illuminate\Foundation\Testing\RefreshDatabase;
```

```
use Laravel\Passport\Passport;
```

```
use Tests\TestCase;
```

```
class AuthTest extends TestCase
```

```
{
```

```
    use RefreshDatabase;
```

```
    protected function setUp(): void
```

```
    {
```

```
        parent::setUp();
```

```
        // Passport needs its encryption keys – use in-memory keys for tests
```

```
        \Artisan::call('passport:keys', ['--force' => true]);
```

```
    }
```

```
    public function test_user_can_login_with_valid_credentials(): void
```

```
    {
```

```
        $user = User::factory()->create(['password' => bcrypt('secret')]);
```

```
        $this->postJson('/api/auth/login', ['email' => $user->email, 'password' => 'secret']  
            ->assertOk()
```

```
            ->assertJsonStructure(['data' => ['token', 'user', 'expires_at']]);
```

```
    }
```

```
    public function test_login_fails_with_wrong_password(): void
```

```
    {
```

```
        $user = User::factory()->create();
```

```
        $this->postJson('/api/auth/login', ['email' => $user->email, 'password' => 'wrong']  
            ->assertStatus(422);
```

```
    }
```

```
    public function test_inactive_user_cannot_login(): void
```

```
    {
```

```
        $user = User::factory()->create(['is_active' => false, 'password' => bcrypt('secret')]);
```

```
        $this->postJson('/api/auth/login', ['email' => $user->email, 'password' => 'secret']  
            ->assertStatus(422);
```

```
    }
```

```
}
```

php

```
<?php
```

```
namespace Tests\Feature;
```

```
use App\Enums\UserRole;
```

```
use App\Models\Clinic;
```

```
use App\Models\User;
```

```
use Illuminate\Foundation\Testing\RefreshDatabase;
```

```
use Laravel\Passport\Passport;
```

```
use Tests\TestCase;
```

```
class RbacTest extends TestCase
```

```
{
```

```
    use RefreshDatabase;
```

```
    protected function setUp(): void
```

```
    {
```

```
        parent::setUp();
```

```
        \Artisan::call('passport:keys', ['--force' => true]);
```

```
    }
```

```
    public function test_doctor_cannot_access_super_admin_routes(): void
```

```
    {
```

```
        $clinic = Clinic::factory()->create();
```

```
        $doctor = User::factory()->create(['role' => UserRole::Doctor, 'clinic_id' =>
```

```
        // Passport::actingAs is the correct way to authenticate in Passport tests
```

```
        Passport::actingAs($doctor, [$doctor->role->value]);
```

```
        $this->getJson('/api/super-admin/clinics')->assertForbidden();
```

```
    }
```

```
    public function test_assistant_cannot_access_doctor_routes(): void
```

```
    {
```

```
        $clinic = Clinic::factory()->create();
```

```
        $assistant = User::factory()->create(['role' => UserRole::Assistant, 'clinic_id' =>
```

```
        Passport::actingAs($assistant, [$assistant->role->value]);
```

```
        $this->getJson('/api/doctor/schedules')->assertForbidden();
```

```
    }
```

```
    public function test_super_admin_can_access_all_clinics(): void
```

```
    {
```

```
        $admin = User::factory()->create(['role' => UserRole::SuperAdmin, 'clinic_id' =>
```

```
        Clinic::factory()->count(3)->create());
```

```
        Passport::actingAs($admin, ['super_admin']);

        $this->getJson('/api/super-admin/clinics')
            ->assertOk()
            ->assertJsonStructure(['data']);
    }
}
```

Updated Deployment Checklist

```
bash

# Install dependencies
composer install --no-dev --optimize-autoloader

# Run all migrations (schema + indexes)
php artisan migrate

# Generate Passport encryption keys (once per environment)
php artisan passport:install --uuids

# Create the personal access client (Passport requires this for createToken())
php artisan passport:client --personal --name="Clinic API Personal"

# Seed the database
php artisan db:seed

# Optimize for production
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan event:cache

# Format code
./vendor/bin/pint

# Run the full test suite
php artisan test

# Start Supervisor (manages all queue workers + scheduler)
supervisorctl reread
supervisorctl update
supervisorctl start all
```

Architecture Change Summary

| BEFORE | AFTER |
|----------------------------|--|
| Sanctum (opaque DB tokens) | → Passport JWT (scoped, expiring) |
| No booking lock | → Cache::lock() per slot |
| N+1 on relations | → with('relation:id,col1,col2') |
| SELECT * on list queries | → Explicit select([...]) |
| 5 DB queries on dashboard | → 1 raw aggregate query + Redis |
| LIKE '%x%' patient search | → MySQL FULLTEXT + whereFullText() |
| No schedule cache | → Tagged Redis, 1h TTL |
| Scattered Cache::forget() | → CacheService owns all keys/TTLs |
| No async side effects | → Jobs: confirm, remind, warm, stats, invalidate |
| No DB transactions | → DB::transaction() on all multi-step writes |
| Clinic check on every req | → Per-user clinic_active cache (5 min) |